

Tema 1 — Plataforma de desarrollo Python

Laboratorio Guiado

Objetivos	Preparar un entorno Python funcional para trabajar desde terminal, Visual Studio Code y notebooks.
Material necesario	Terminal, Python 3, pip, venv, Visual Studio Code con extensiones Python y Jupyter.

Laboratorio 1. Verificación inicial de la plataforma

1. Abrir una terminal del Sistema.
2. Comprobar sistema, usuario y directorio de trabajo.

```
cat /etc/os-release
uname -r
whoami
pwd
```

3. Comprobar el intérprete Python y pip disponibles.

```
python3 --version          # resultado python 3.9.25 que es la version del sistema
python3.13 --version       # resultado python 3.13.13
ls -l /usr/bin/python*.    # El Sistema apunta a python 3.9
```

Resultado esperado: identificar la versión de Python del Sistema, las versiones de python disponibles, y la ruta de los ejecutables.

Laboratorio 2. Crear estructura de proyecto del curso

1. Descarga el repositorio de Git para los laboratorios del curso. Encontrarás el Directorio con los laboratorios del Tema 1.

```
git clone https://github.com/IconoTC/Programaci-n-PYTHON-AF-75813-Grupo-97783.git Curso_Python
cd ~/Curso_Python/
pwd
```

2. Crear la estructura mínima del proyecto.

```
tree .
cat README.md
ls -la
```

Resultado esperado: el proyecto contiene carpetas separadas para scripts y notebooks, más un **README.md** para documentar comandos y evidencias.

Laboratorio 3. Crear y activar el entorno virtual

1. Crear un entorno virtual en la raíz del proyecto.

```
python3.13 -m venv .venv --prompt "Curso_Python"
```

2. Activar el entorno virtual.

```
source .venv/bin/activate
```

3. Verificar que Python y pip apuntan al entorno del proyecto.

```
python --version
which python
python -m pip --version
```

4. Actualizar pip dentro del entorno virtual e instalar el kernel y las dependencias para ejecutar los Jupyter Notebooks

```
python -m pip install --upgrade pip
python -m pip install notebook ipykernel
```

Nota: durante el curso se usará un entorno virtual dentro del Directorio del repositorio del curso para evitar mezclar con el entorno python del Sistema.

Laboratorio 4. Uso de Python en una terminal

1. Este es el primer script que vamos a ejecutar. Está ubicado en: ~/Curso_Python/Tema01/src/holamundo.py :

```
import os
import sys
import platform

print("Entorno Python operativo")
print("Usuario:", os.getenv("USER"))
print("Directorio actual:", os.getcwd())
print("Ejecutable:", sys.executable)
print("Version:", sys.version.split()[0])
print("Sistema:", platform.platform())
```

2. Ejecutar el script desde la raíz del Proyecto y comprobar el resultado.

```
cat Tema01/src/holamundo.py
python Tema01/src/holamundo.py
```

3. En la misma terminal, ejecutar python de forma interactiva. Con el prompt de python (>>>) ejecuta varios comandos de forma interactiva

```
python
>>> print("Hola Mundo")
>>> valores=(1,2,3,4)
>>> print(valores)
>>> CTRL+D # Sale de python interactivo
```

Laboratorio 5. Usar Visual Studio Code

1. Asegúrate que sigues ubicado en el Directorio del curso y ejecuta Visual Studio Code desde ahí.

```
pwd      # Si no estas en /home/user/Curso_Python cambia a ese directorio
code .   # Se abre Visual Studio Code
```

2. Abrir una terminal integrada en VS Code (Barra de menús > Terminal > New Terminal) y comprobar que el entorno está activo.

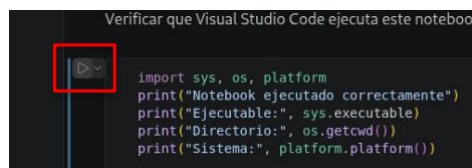
```
pwd          # /home/user/Curso_Python
which python  # Debe responder con ~/Curso_Python/.venv/bin/python (Python local)
python --version # python 3.13.13
```

3. En el árbol de la izquierda, seleccionar el fichero de Código ~/Curso_Python/Tema01/src/holamundo.py . En la parte superior derecha encuentras el botón Run.  Púlsalo y esto ejecuta el script en la terminal que tengas abierta o en una nueva terminal. Comprueba el resultado

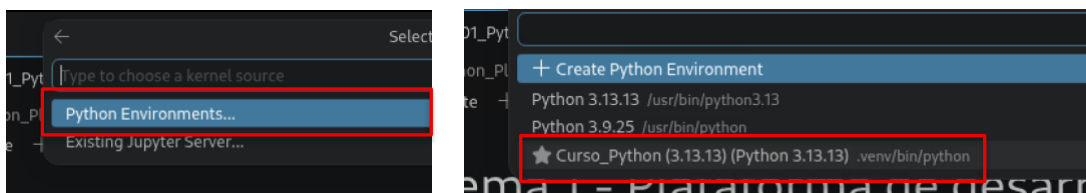
Resultado esperado: VS Code debe ejecutar Python usando el intérprete del proyecto, no el Python global del sistema. Puedes usar Visual Studio Code para ejecutar el Código, o una terminal de linux con el Entorno virtual activado

Laboratorio 6. Primer notebook en Visual Studio Code

1. Dentro de Visual Studio Code, seleccionar el notebook
Curso_Python/Tema01/notebooks/01_Plataforma_Desarrollo.ipynb.
2. Verás celdas Markdown (Texto) y celdas Code (Código). Las celdas Markdown son explicativas y las de Código son Código que puedes ir ejecutando por pasos. También puedes editar y modificar las celdas de Código para hacer diferentes pruebas.
3. Selecciona en la parte izquierda de la primera celda de Código el botón play



4. En la barra superior te pedirá seleccionar el kernel para ejecutar python. Selecciona el kernel del venv (2 pasos).



5. Debajo del cuadro de Código recibirás el resultado de la ejecución. Puedes seguir ejecutando el siguiente cuadro y así sucesivamente. Ten en cuenta que en futuros notebooks, todos los bloques de Código en un mismo notebook están relacionados como si fueran un único script. Si defines variables, funciones, etc en un Bloque, quedan definidas para los siguientes bloques.
6. Puedes probar a crear nuevas celdas Markdown y Code en este notebook y probarlas con los botones +Code y +Markdown. 

7. Por ejemplo, prueba a crear una celda Code e inserta el siguiente código

```
import sys, os, platform
print("Notebook ejecutado correctamente")
print("Ejecutable:", sys.executable)
print("Directorio:", os.getcwd())
print("Sistema:", platform.platform())
```

Resultado esperado: Pulsando el botón de run en la celda de Código, el notebook ejecuta las celdas usando el mismo entorno virtual que el proyecto.

Laboratorio 7. Instalar dependencias y generar requirements.txt

1. Cierra Visual Studio Code y Vuelve a la Terminal. Instala un modulo de prueba dentro del entorno virtual.

```
pip install cowsay
```

2. Comprobar que el paquete está instalado en el entorno activo.

```
pip show cowsay
pip list          # Muestra todos los modulos instalados con pip
cowsay -t "Hola Mundo"
```

3. Generar el archivo de dependencias del proyecto.

```
pip freeze > requirements.txt
cat requirements.txt
```

Laboratorio 8. Reconstrucción controlada del entorno

1. Desactivar el entorno virtual.

```
deactivate
```

2. Eliminar el entorno virtual y reconstruirlo desde requirements.txt.

```
rm -rf .venv
python3.13 -m venv .venv --prompt "Curso_Python"
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

3. Ejecutar de nuevo el script y el notebook para comprobar que el proyecto es reproducible.

```
cowsay -c tux -t "hola"
python Tema01/src/holamundo.py
```

Ejercicios propuestos

1. Crear un segundo script Tema01/src/info_entorno.py que muestre ruta del ejecutable, versión de Python, directorio actual y sistema operativo.
2. Añadir al README.md las instrucciones para crear, activar y reconstruir el entorno virtual.
3. En una Ventana de shell con el entorno active, desactivar el entorno virtual, comprobar que Python se usa fuera del entorno y volver a activarlo.

Guía de resolución de problemas

Problema	Diagnóstico	Solución
No aparece (Curso_Python) en el prompt	El entorno no está activado	Ejecutar source .venv/bin/activate desde la raíz del proyecto.
python usa una ruta inesperada	El intérprete seleccionado no es el del proyecto	Comprobar which python y seleccionar el intérprete .venv en VS Code.
pip instala paquetes fuera del proyecto	Se está usando pip del sistema	Usar python -m pip y confirmar que el entorno virtual está activo.
VS Code no ejecuta el notebook	Kernel o extensión no seleccionados	Seleccionar el kernel del entorno .venv y revisar la extensión Jupyter.
ModuleNotFoundError con requests	Dependencia no instalada en el entorno activo	Activar .venv y ejecutar python -m pip install -r requirements.txt.